

AI ASSISTED CODING

Assignment-8

2303A51463

B-07

Test-Driven Development with AI – Generating and Working with Test Cases

Task 1 – Test-Driven Development for Even/Odd Number Validator

Prompt:

Generate test cases for a Python function `is_even(n)` that validates integer input and handles zero, negative numbers, and large integers.

[25]
✓ 0s

```
def is_even(n):  
    if not isinstance(n,int):  
        raise TypeError("Input must be an integer")  
    return n%2==0  
  
print(is_even(2))  
print(is_even(7))  
print(is_even(0))  
print(is_even(-4))  
print(is_even(9))
```

▼

```
... True  
... False  
... True  
... True  
... False
```

Task 2 – Test-Driven Development for String Case Converter

Prompt:

Generate test cases for two functions `to_uppercase(text)` and `to_lowercase(text)` that handle empty strings, mixed-case input, and invalid inputs such as numbers or `None`.

[26]

✓ 0s

```
def to_uppercase(text):
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text.upper()

def to_lowercase(text):
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text.lower()

print(to_uppercase("ai coding"))
print(to_lowercase("TEST"))
print(to_uppercase(""))
```

✓

... AI CODING
test

Task 3 – Test-Driven Development for List Sum Calculator

Prompt:

Generate test cases for a function `sum_list(numbers)` that handles empty lists, negative numbers, and safely ignores non-numeric values.

[27]

✓ 0s

```
def sum_list(numbers):
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")
    total = 0
    for item in numbers:
        if isinstance(item, (int, float)):
            total += item
    return total

print(sum_list([1, 2, 3]))
print(sum_list([]))
print(sum_list([-1, 5, -4]))
print(sum_list([2, "a", 3]))
```

✓

6
0
0
5

Task 4 – Test Cases for Student Result class

Prompt:

Generate test cases for a StudentResult class with methods add_marks(mark), calculate_average(), and get_result(). Marks must be between 0 and 100, and the result should be Pass if average ≥ 40 , otherwise Fail.

[28]
✓ 0s

```
class StudentResult:
    def __init__(self):
        self.marks=[]

    def add_marks(self,mark):
        if not isinstance(mark,(int,float)):
            raise TypeError("Mark must be numeric")
        if mark<0 or mark>100:
            raise ValueError("Mark must be between 0 and 100")
        self.marks.append(mark)

    def calculate_average(self):
        if not self.marks:
            return 0
        return sum(self.marks)/len(self.marks)

    def get_result(self):
        avg=self.calculate_average()
        return "Pass" if avg>=40 else "Fail"

student=StudentResult()
student.add_marks(60)
student.add_marks(70)
student.add_marks(80)
print(student.calculate_average())
print(student.get_result())
```

▼ ... 70.0
Pass

Task 5 – Test-Driven Development for Username Validator

Prompt:

Generate test cases for a function `is_valid_username(username)` with rules: minimum 5 characters, no spaces, and only alphanumeric characters.

[30]

✓ 0s

```
def is_valid_username(username):
    if not isinstance(username, str):
        return False
    if len(username) < 5:
        return False
    if " " in username:
        return False
    if not username.isalnum():
        return False
    return True

print(is_valid_username("user01"))
print(is_valid_username("ai"))
print(is_valid_username("user name"))
print(is_valid_username("user@123"))
print(is_valid_username(12345))
print(is_valid_username(None))
```

▼

```
... True
False
False
False
False
False
```

